Webhook

11 August 2026 10:23 AM

🔔 What is a Webhook?

A **webhook is a way for one system to automatically notify another system when something happens.**
Think of it like this:

> **"Don't keep asking me whether something happened. I'll call you when it happens."**

🏦 Simple Payment Example

Suppose you have an e-commerce application using Stripe for payments.
A customer pays ₹1,000.
Your system needs to:

1. Mark the order as **Paid**
2. Send a receipt
3. Tell the fulfillment service to ship the order

But here's the problem:
Your application doesn't know exactly when Stripe confirms the payment.
There are two ways to find out.

❌ Approach 1: Polling

Your application keeps asking Stripe:

Your System → Stripe: Payment successful?

Stripe → Your System: No

5 seconds later...
Your System → Stripe: Payment successful?

Stripe → Your System: No

5 seconds later...
Your System → Stripe: Payment successful?

Stripe → Your System: YES!
This is called **polling**.

Problems

Imagine checking every 5 seconds.
Most requests will say:

"No, nothing happened."
You're wasting:

- Network requests
- Server resources
- API quota

And there is also a delay.
If payment happens at:

10:00:01
but you check at:

10:00:05
you only learn about it at 10:00:05.

☑ Approach 2: Webhook

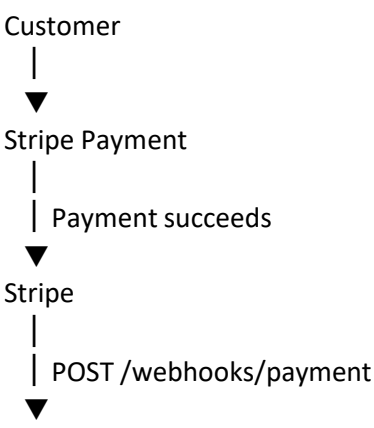
Instead, you tell Stripe:

"When the payment succeeds, call this URL."

For example:

https://myshop.com/webhooks/payment

Now the flow becomes:



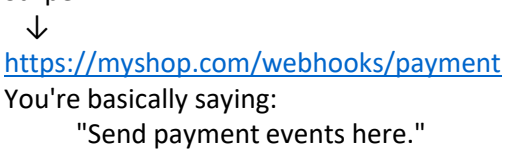
Stripe **pushes** the information to you.
Your system doesn't have to keep asking.

🔄 How a Webhook Works

There are basically **4 steps**.

1. Registration

You give the provider your webhook URL.



You're basically saying:

"Send payment events here."

2. Event Happens

Something happens in the provider.
For example:

payment_intent.succeeded
Meaning:

Payment was successful.

3. Provider Sends HTTP Request

Stripe sends a POST request to your server.
Something like:

POST /webhooks/payment
Content-Type: application/json
With data:

```
{
  "id": "evt_001",
  "type": "payment_intent.succeeded",
  "orderId": "order_42",
  "amount": 1000
}
```

Now your application knows:

"Payment succeeded for order 42."

4. Your Server Responds

Your server says:

200 OK

This basically means:

"I received your webhook."

🔐 Why Does the Webhook Have a Signature?

Here's an important system-design question.

Anyone on the internet could potentially send:

```
POST /webhooks/payment
```

and say:

```
{
  "payment": "successful"
}
```

You obviously **cannot trust that**.

So the provider signs the webhook.

```
Provider
   │
   │ Payload + Signature
   ▼
Your Server
   │
   ▼
Verify Signature
```

If the signature is valid:

☑ Probably from the provider

If invalid:

❌ Reject it

This prevents someone from simply pretending to be Stripe/GitHub/etc.

3. Anatomy of a Webhook Request

A webhook is usually an **HTTP POST request** sent by a provider to your application's webhook endpoint.

Think of it as:

```
Provider
   │
   │ POST
   │
```

/webhooks/payment


A webhook request mainly has **3 parts**:

1. **Endpoint URL**
2. **Request Headers**
3. **Request Body**

1. Endpoint URL

The endpoint is the URL where your application receives the webhook.
For example:

https://myshop.com/webhooks/payment
You give this URL to the provider.
When something happens:

Payment succeeds


Stripe


POST /webhooks/payment


Your Server
So the endpoint is basically:

> **"Where should the provider send the event?"**

2. Request Headers

Headers contain **additional information about the webhook request**.
For example:

Content-Type: application/json
Stripe-Signature: ...
Different providers use different header names.

Common information found in headers:

Header	Purpose
Content-Type	Tells you the format of the body
Event type	Tells you what happened
Delivery ID	Identifies the event/delivery
Signature	Helps verify the sender
Timestamp	Helps detect replayed requests
User-Agent	Identifies the provider

Content-Type

Usually:

Content-Type: application/json
This means:

> "The webhook body contains JSON."

Event Type

The provider may tell you what happened.
For example:

payment_intent.succeeded
or:

pull_request
Your application can then decide what to do.

payment_intent.succeeded
↓
Mark order as paid

Delivery ID

The provider usually gives the event/delivery a unique ID.
Example:

evt_12345
This is **very important for duplicate detection**.
Suppose you receive:

evt_12345
evt_12345
Your system can check:

Have I already processed evt_12345?
YES → Ignore it
NO → Process it
This helps make webhook processing **idempotent**.

Signature

The provider may sign the webhook using a secret.
Example:

Stripe-Signature: ...
Your server verifies the signature before trusting the request.

Webhook
↓
Verify Signature
↓
Valid? ── No → Reject
|
Yes
↓
Process Event
This helps ensure that the request really came from the provider and wasn't modified.

Timestamp

A timestamp tells you **when the webhook was generated/sent**.
It can also be used together with a signature to detect **replayed requests**.
For example, an attacker might capture a valid webhook and try sending the exact same request later.

The timestamp can help your application reject old/replayed requests according to the provider's security model.

3. Request Body

The **body contains the actual event data**.
Usually it is JSON:

```
{
  "id": "evt_123",
  "type": "payment_intent.succeeded",
  "data": {
    "orderId": "order_42",
    "amount": 1000,
    "currency": "USD"
  }
}
```

The body usually contains two important things:

Event Envelope

Tells you:

What happened?

Example:

```
{
  "type": "payment_intent.succeeded"
}
```

Resource Data

Tells you:

What object was affected?

Example:

```
{
  "orderId": "order_42",
  "amount": 1000
}
```

So you can think:

```
Webhook Body
├── Event information
│     └── What happened?
│
└── Resource information
      └── What was affected?
```

Example: Payment Webhook

Imagine Stripe sends:

```
POST /webhooks/payment
Content-Type: application/json
Stripe-Signature: abc123
Body:

{
  "id": "evt_123",
  "type": "payment_intent.succeeded",
  "data": {
```

```
  "orderId": "order_42",
  "amount": 1000,
  "currency": "USD"
 }
}
```

Your application processes it:

```
Receive webhook
    ↓
Verify signature
    ↓
Check event ID
    ↓
Check event type
    ↓
Save event
    ↓
Queue processing
    ↓
Mark Order as PAID
```

4. Building a Webhook Receiver

A **webhook receiver** is an endpoint that accepts webhook requests from another system.
In production, the main challenges are:

- Duplicate events
- Events arriving out of order
- Retries
- Fake requests
- Slow or failed services

Dedicated Endpoint

Use a separate endpoint for each provider:

```
POST /webhooks/stripe
POST /webhooks/github
POST /webhooks/ai
```

Each provider can have different:

- Signature rules
- Headers
- Event formats
- Retry behavior

Using separate endpoints keeps the system easier to manage.

Verify Before Parsing Business Data

Verify the webhook before trusting its data.
Usually this is done using an **HMAC signature**.

```
Webhook
  ↓
Verify Signature
  ↓
Valid?
  ┌──┴──┐
No  Yes
↓   ↓
Reject Process
```

Important:

- Verify the **raw request body**.
- Check the timestamp if provided.
- Reject old/replayed requests.
- Store webhook secrets securely.

Make Processing Safe to Repeat

Webhooks commonly use **at-least-once delivery**, meaning the same event can arrive more than once.
Example:

evt_123 → received
evt_123 → received again
Store the event ID and check whether it was already processed.

Already processed?
YES → Ignore duplicate
NO → Process
This is called **idempotent processing**.

Do Not Depend on Delivery Order

Events may arrive in a different order than they were created.
Example:

Created → Paid → Shipped
could arrive as:

Paid → Created → Shipped
For important operations, check the **latest state from the provider** instead of blindly trusting event order.

Return the Right Status Code

Status	Meaning
200 / 204	Event accepted
400	Invalid request
401 / 403	Signature/authentication failed
404	Endpoint not found
429	Too many requests
500 / 503	Temporary server failure

Important: Don't return 200 OK until the event has been safely saved or queued.

5. Webhook Security

Webhook endpoints are exposed to the internet, so they must be treated like public APIs.

Verify Signatures

Signature verification prevents attackers from sending fake events.
Without it, someone could send:

payment_succeeded
and your system might incorrectly mark an order as paid.

Protect Against Replay

A valid webhook could be sent again later.
Use:

- Timestamp checking
- Event/delivery ID
- Duplicate detection
- Idempotent processing

This is especially important for payments, access control, emails, and external jobs.

Use IP Allow Lists Carefully

Some providers publish their IP addresses.
You can allow requests only from those IPs, but:

IP allow lists should not replace signature verification.

IP ranges can change.

Avoid Sensitive Data Leaks

Don't log:

- Secrets
- Tokens
- Authorization headers
- Full payment information
- Sensitive personal data

Also, don't put secrets in webhook URLs.

6. Designing Scalable Webhook Infrastructure

A simple webhook handler may work for small traffic.
At large scale, separate **receiving** from **processing**.

Provider
↓
Webhook Endpoint
↓
Verify Signature
↓
Detect Duplicate + Save
↓
Queue
↓
Worker
↓
Business Logic

Keep Receiving Fast

The webhook endpoint should mainly:

1. Verify signature
2. Validate event
3. Check duplicate
4. Save event
5. Put event into queue
6. Return 2xx

Heavy work should happen in the background.

Queue

A queue helps handle traffic spikes.

10,000 Webhooks
↓
Queue
↓
Workers

Common choices:

- SQS
- RabbitMQ
- Kafka

- Google Cloud Pub/Sub
- Azure Service Bus

Store Events for Audit and Replay

Store information such as:

Provider
Event ID
Event Type
Delivery ID
Body
Processing Status
Timestamps
This helps answer:

- Did we receive the event?
- Did verification fail?
- Was it processed?
- Why did it fail?
- Can we replay it?

Process with Workers

Workers perform the actual business logic:

Worker
├── Update database
├── Call APIs
├── Send notifications
├── Update order
└── Trigger workflows
Workers allow you to control how much processing happens at the same time.

Retry with Backoff and Jitter

Temporary failures should be retried.
Example:

1 sec → 2 sec → 4 sec → 8 sec
Backoff = wait longer between retries.
Jitter = add some randomness so many retries don't happen at exactly the same time.
Not every error should be retried.

Use a Dead Letter Queue

If an event keeps failing:

Event
↓
Retry
↓
Retry
↓
Still fails
↓
DLQ
DLQ = Dead Letter Queue
It stores failed events so developers can investigate and replay them later.

Add Observability

Monitor:

- Events received

- Signature failures
- Queue size
- Processing delay
- Retry count
- DLQ count
- API failure rate

Use IDs such as:

Event ID
Correlation ID
Resource ID
to trace requests through the system.

7. Common Mistakes

Avoid:

1. **Doing too much work in the HTTP handler**
2. **Skipping signature verification**
3. **Losing the raw request body before verification**
4. **Assuming exactly-once delivery**
5. **Assuming events arrive in order**
6. **Returning 200 before saving the event**
7. **Subscribing to unnecessary event types**
8. **Having no reconciliation/recovery mechanism**

8. When to Use Webhooks

Use webhooks when:

- Another system owns the event.
- You need to react quickly.
- Polling would waste resources.
- You can provide a reliable endpoint.
- You can handle retries and duplicates.

Don't depend only on webhooks when:

- The receiver isn't reliably reachable.
- Strict event ordering is required.
- You need complete control over processing rate.
- There is no replay/reconciliation mechanism.
- The provider cannot reliably identify or sign events.